TASK 7 – Build a Module

Module organisation and imports

Save the rate constants and calculation functions in milkrate.py. A separate script imports and uses them. import milkrate accesses names through milkrate.payout; from milkrate import payout imports the selected name directly; import milkrate as mr gives the module a short alias and calls mr.payout(). The first style makes the source module explicit and reduces name collisions.

Main guard and module search

if __name__ == "__main__": runs the demonstration only when milkrate.py is executed directly. When another script imports it, __name__ is "milkrate", so the demonstration does not run. Without the guard, top-level demonstration code executes during import. Python searches the current script directory and locations listed in sys.path, including installed package locations. Standard modules include decimal for decimal arithmetic and math for mathematical functions.

Completed milkrate.py

```
from decimal import Decimal, ROUND_HALF_UP

"""Rate card and payment functions for the village milk co-operative."""
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP = Decimal("0.1")

def rate_per_litre(fat, snf):
    """Return the rate per litre rounded to two decimal places."""
    fat_steps = ((Decimal(str(fat)) - Decimal("3.5")) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((Decimal(str(snf)) - Decimal("8.5")) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Return the amount payable, rounded to two decimal places."""
    return (Decimal(str(litres)) * rate_per_litre(fat, snf)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Separate test script

```
import milkrate as mr

print(mr.payout("8.5", "4.2", "8.7"))
```

Expected demonstration

10 L fat 3.5 snf 8.5 rate 30.00 pay 300.00
8.5 L fat 4.2 snf 8.7 rate 33.40 pay 283.90
12 L fat 3.1 snf 8.3 rate 27.80 pay 333.60

The demonstration table is not printed on import because the main guard is false while the module is imported. The imported functions remain available.

Alternative and complexity

Copying functions into each script duplicates logic. If a rate changes, every copy must be located and updated; a module keeps one authoritative implementation. The first import reads and executes the module once in the process and stores it in sys.modules. Later imports normally reuse the loaded module. Importing has a one-time start-up cost; each subsequent function call has its ordinary call cost. Calculation for n entries remains O(n) time. The module itself uses O(1) configuration space.